Head in the Clouds

A trekking management application. Cloud Computing and DevOps (CCDL) project report, academic year 2026-27

Students	Navon John Lukose (240905602), Srishti Sudesh (240905626)
Section and batch	CSE C, batch 2
Institute	Manipal Institute of Technology
Department	School of Computer Engineering
Supervisor	Mohandas Shenoy P
Repository	https://github.com/Navonjl/mad2-trekking-management (private)

Problem statement

Trekking groups run their trips on spreadsheets and phone calls, which makes trek approvals, registrations, slot availability and booking records hard to keep straight. The application gives three kinds of user one shared set of data with different permissions over it. An admin creates trek routes, adds staff, assigns them to treks, and oversees every user and booking. Trek staff run only the treks assigned to them, setting slots and status and seeing who has signed up. Trekkers register themselves, search what is on offer, book a place, and keep a history of where they have been.

The load is seasonal. A trekking operator sees almost nothing for weeks and then a burst when a popular route opens, so a fixed server is either wasted or too small. That is why the application is built to run on managed Azure services and be scheduled by Kubernetes instead of being pinned to one host.

Approach

The application is a JSON API with a separate single-page front end. The same data has to appear three different ways, so putting every rule in the API means the browser cannot skip a check, and the three dashboards become three sets of components reading one set of endpoints.

Five decisions account for most of the design.

The first is a single user table for all three roles. Admin, staff and trekker store the same things and differ only in what they may do, so they share a `user` table with roles in a many-to-many join. Login is then one code path instead of three, and a role check is a decorator on the view. The two fields only staff have, employee code and duty status, sit in a separate `staff_profile` table so they are not NULL for every trekker.

The second is keeping the booking rules as plain conditions. The four rules, that the trek must exist, be Open, have a free slot, and not already be booked by this user, are four numbered `if` statements in `book_trek()`.

The third is the one exception to that. Taking a slot is not a plain `if`. Reading the count, checking it, then subtracting one is three steps, so two people pressing Book

on the last slot can both pass the check. Instead a single conditional `UPDATE` decrements the count only while a slot is free and reports how many rows it changed. Zero rows means the trek filled up first. An in-process lock would have been enough on one server and is useless here, because the API runs as several pods behind a load balancer and the only thing they share is the database. The check has to happen where the data is.

The fourth is storing `available_slots` instead of counting bookings. Counting would mean a `COUNT` per trek on every listing. The stored number changes in the same request that creates the booking, so the two cannot drift.

The fifth is that every API pod is disposable. No session state, no uploaded file and no scheduled work lives on a pod's filesystem. Authentication is a bearer token, generated CSV exports go to Blob Storage, mail goes out through Azure Communication Services, and the cache is a shared Azure Managed Redis instance. A rolling deploy, a scale-in or a node upgrade can kill a pod at any moment, and nothing is lost with it.

Cloud architecture

Everything runs in the Central India region in one virtual network spread across three availability zones. The AKS nodes, the Application Gateway for Containers frontend and the PostgreSQL server each get their own subnet, and Redis and Key Vault are reached through private endpoints, so none of the data services has a public address.

Concern	Azure service	Why
Relational data	Azure Database for PostgreSQL flexible server, zone-redundant HA	the booking decrement needs a real transactional database, and the standby in another zone takes over on its own
Cache	Azure Managed Redis	shared by every API pod, so a listing cached by one is a listing served by all of them, and it doubles as the Celery result backend
Job queue	Azure Service Bus	a managed broker with a built-in dead-letter subqueue, so a burst of exports cannot be lost with the pod that was holding them
Job scheduling	Azure Functions, timer trigger	fires the daily and monthly jobs on a cron expression, with no long-running scheduler process to keep alive
Compute	Azure Kubernetes Service	one cluster runs the API and the workers, with node capacity managed by node auto-provisioning
Container images	Azure Container Registry	private registry attached to the cluster, which pulls through its managed identity with no pull secret
DNS	Azure DNS	the zone for the public hostname, pointing at the Front Door endpoint
Edge and filtering	Azure Front Door Premium with a WAF policy	TLS termination and caching at the edge, plus rate-limit rules on login and booking
Ingress	Application Gateway for Containers	a managed layer 7 load balancer configured from Gateway API objects by the ALB Controller running in the cluster
[illegible]	[illegible] Insights, [illegible]	[illegible] and [illegible] so no

A request for a page reaches Azure DNS, then Front Door, and is served from Blob Storage. A request to `/api` passes the WAF policy, goes to Application Gateway for Containers, then to the API service, then to a pod. That pod reads the trek listing from Redis if it is warm and from PostgreSQL if it is not, writes through to PostgreSQL, and pushes any slow work onto Service Bus. Because the front end and the API are served from one Front Door endpoint, requests stay same-origin and no CORS configuration is needed.

The WAF rules each have a job. The two endpoints worth protecting are login, where a rate-limit rule slows credential stuffing, and booking, where the interesting abuse is a script hammering `POST /api/bookings` the second a popular trek opens. The database is already correct under that load, but the rule keeps one client from spending the capacity everyone else is queuing for.

Kubernetes deployment

The cluster runs three workloads in one namespace.

Workload	Kind	Replicas	Notes
`trek-api`	Deployment	2 to 10	Flask under gunicorn, scaled by a HorizontalPodAutoscaler on CPU, with a readiness probe on `/api/health` so a pod takes traffic only once it can reach PostgreSQL
`trek-worker`	Deployment	2 to 6	Celery workers consuming Service Bus, scaled on queue length by KEDA's Service Bus scaler
`db-migrate`	Job	per release	runs the Alembic migration once before the new API image is rolled out, as a Helm pre-upgrade hook

A PodDisruptionBudget keeps at least one API pod up during node drains, and requests and limits are set on every container so node auto-provisioning can pick VM sizes and one runaway worker cannot starve the API. Nothing mounts a PersistentVolume, which is what makes the rolling deploy safe.

There is deliberately no Celery beat pod. Beat is a singleton by design, since a second scheduler fires every daily reminder twice, so it cannot be scaled and

cannot be rolled without a gap. Moving the schedule out of the cluster removes the singleton. A timer-triggered Azure Function calls `send_task`, which puts an ordinary message on Service Bus for any worker to pick up. Azure keeps the scheduler running, and every workload left in the cluster is horizontally scalable.

Infrastructure as code and CI/CD

The Azure side is Terraform with the azurerm provider. It covers the virtual network and subnets, the AKS cluster with its add-ons, the PostgreSQL server, Azure Managed Redis, the Service Bus namespace and queue, the Functions app, the storage accounts, Front Door and its WAF policy, the DNS zone, the container registry, Key Vault, the managed identities and their federated credentials, and the monitoring resources. State lives in a blob container in a separate storage account, and Terraform takes a lease on the state blob during each apply, so two people cannot apply at once. The cluster side is a Helm chart holding the three workloads above, with one `values.yaml` per environment, so staging and production differ by a file and not by a hand-edited manifest.

Terraform suits things with a lifecycle measured in months and is awkward for things that redeploy several times a day, and Helm is the reverse, so each tool gets the half it is good at. Ansible was considered and left out. AKS manages the node images and upgrades, which leaves nothing to configure over SSH, and a second tool that also claims to own the cluster invites drift.

GitHub Actions is the pipeline. It signs in to Azure with an OIDC federated credential, so the repository holds no client secret. A push runs the test suite and builds the frontend. A merge to `main` builds the API image, pushes it to the container registry under the commit SHA, and runs `helm upgrade` against staging. Production is the same job behind a manual approval. Images are tagged by SHA and never by `latest`, so a rollback is `helm rollback` to a tag that still exists.

Frameworks and libraries

Layer	Technology	Version
API	Flask	3.1.3
ORM	Flask-SQLAlchemy / SQLAlchemy	3.1.1 / 2.0.51
Database	Azure Database for PostgreSQL flexible server	17
Auth	Flask-Security-Too	5.8.1
Cache	Flask-Caching on Azure Managed Redis	2.4.1
Jobs	Celery, brokered on Azure Service Bus	5.6.3
Scheduling	Azure Functions, timer trigger	
UI	Vue 3 with Vue Router	3.5.40 / 5.2.0
Styling	Bootstrap	5.3.8
Charts	Chart.js	4.5.1
Build	Vite	8.1.5
Container runtime	Docker, images on Azure Container Registry	
Orchestration	Azure Kubernetes Service	
Packaging	Helm	4
Provisioning	Terraform with the azurerm provider	1.16 / 5
Pipeline	GitHub Actions	

Configuration is read from the pod environment. In development that is a local `.env` file parsed by an eight-line function in `app/config.py`. In the cluster the same variables come from Key Vault through the Secrets Store CSI driver, which syncs them into a Kubernetes Secret, so the application code cannot tell the difference and no secret is baked into an image.

Azure Managed Redis exposes a single database, so the response cache and the Celery result backend share it and are kept apart by key prefix. Its Enterprise clustering policy presents the cluster as one endpoint, which the plain Redis client used by Flask-Caching and Celery expects. The broker lives on Service Bus, so losing the cache costs a few slow page loads and leaves the job queue intact.

Database design

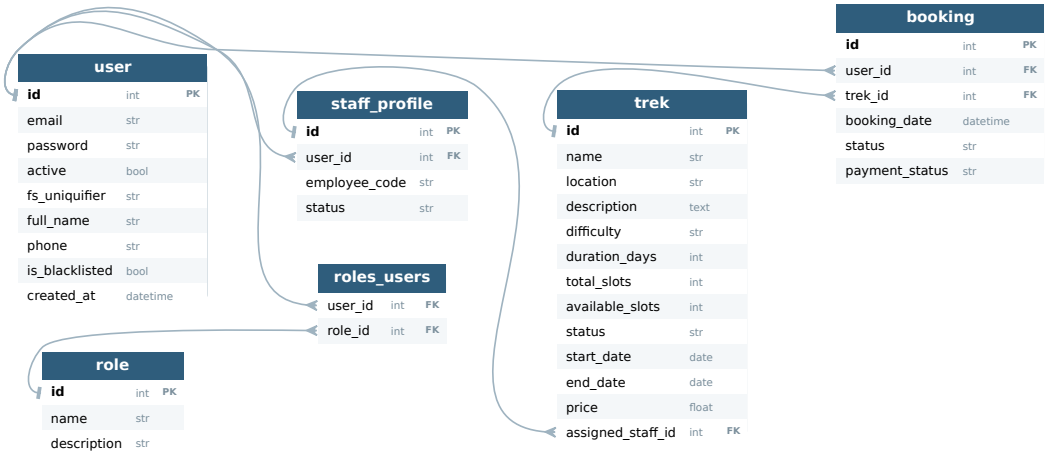


Figure 1: ER diagram

The diagram is generated from the SQLAlchemy models by graphviz, so it matches the schema exactly. Difficulty is Easy, Moderate or Hard. Trek status is Pending, Approved, Open, Closed or Completed. Booking status is Booked, Cancelled or Completed.

The `user` table also carries two-factor and WebAuthn columns supplied by Flask-Security's model mixin, which this project does not use.

Schema changes are applied by an Alembic migration run as the `db-migrate` Job, never by `create_all()` against a live database and never by hand in a GUI tool. Backups are the flexible server's automated backups with point-in-time restore. The database is now the only copy of anything, so the restore window is the setting that matters most in the whole deployment.

API endpoints

Everything answers JSON. Every route except login and register needs an `Authentication-Token` header.

Authentication: `POST /api/accounts/login?include_auth_token`, `POST /api/accounts/logout`, `POST /api/register` (trekkers only), `GET /api/me`, `PUT /api/profile`.

Trekker: `GET /api/treks` (filters `q`, `difficulty`, `location`, `min_days`, `max_days`; cached), `POST /api/bookings`, `GET /api/bookings`, `POST /api/bookings/<id>/cancel`, `POST /api/exports/bookings`, `GET /api/exports/<task_id>`, `GET /api/exports/download/<filename>`.

Admin: `GET /api/admin/stats` (cached); `GET POST /api/admin/treks` and `PUT DELETE /api/admin/treks/<id>`; `PUT /api/admin/treks/<id>/assign-staff`; `GET POST /api/admin/staff`; `GET /api/admin/users`; `POST /api/admin/users/<id>/{deactivate,reactivate,blacklist,unblacklist}`; `GET /api/admin/bookings`; `POST /api/admin/bookings/<id>/mark-paid`.

Trek staff: `GET /api/staff/dashboard`, `PUT /api/staff/treks/<id>`, `GET /api/staff/treks/<id>/participants`. Each of these checks the trek is assigned to the caller and answers 403 otherwise.

`GET /api/health` is unauthenticated and serves the readiness probe and the Front Door origin health probe. It reports failure if the database is unreachable, so a pod that cannot serve is taken out of rotation instead of returning errors.

Deactivating a user blocks login. Blacklisting lets them log in but refuses booking, cancelling and exporting.

Background jobs

Job	Trigger	What it does
`send_daily_reminders`	Azure Functions timer, daily 18:00	messages every trekker booked on a trek starting within three days, with the date and joining instructions. Posts to a Google Chat space through an incoming webhook, or sends e-mail if none is set
`send_monthly_activity_report`	Azure Functions timer, 1st at 06:00	HTML report of the previous month, covering treks conducted, users who took part and the five most popular treks, e-mailed to the admin through Azure Communication Services
`export_bookings_csv`	user, from their dashboard	writes that trekker's history to CSV, uploads it to Blob Storage under a per-user prefix, and alerts them with a SAS link

The Celery instance is created in `app/celery_utils.py` and configured from the `CELERY` block in `app/config.py`. A small `FlaskTask` base class runs each task inside an application context, which is why the tasks use `db.session` directly.

Service Bus makes idempotency a requirement. It delivers at least once, and if a worker pod is evicted partway through an export, the message lock expires and the message is delivered again. Every task is therefore safe to run twice, and the second run of an export overwrites the same blob instead of producing a duplicate. A message that fails past the queue's maximum delivery count moves to the dead-letter subqueue, whose size is one of the alerts below, so a poison message is visible instead of silently retrying forever.

Observability and cost

Container Insights sends pod logs and metrics to a Log Analytics workspace, and PostgreSQL, Redis and Service Bus publish their platform metrics to Azure Monitor.

Four alert rules cover the failures that matter. They fire on the API 5xx rate seen by Application Gateway for Containers, on Service Bus active message count staying high (the workers are not keeping up), on any dead-lettered message, and on PostgreSQL storage use. All four notify one action group, which sends e-mail and posts to the same Google Chat space the reminders use, so there is one place to subscribe instead of an alert configured per service.

On cost, the parts that can scale to nothing do. Node auto-provisioning removes empty nodes, Front Door, Blob Storage and the Functions app are billed by use, and the API scales down to two pods overnight. PostgreSQL and Redis cannot scale to nothing, so they set the floor of the monthly bill and are sized small on purpose, since the seasonal peak is absorbed by the stateless tier and not by the database. The staging PostgreSQL server is stopped when nobody is testing, which the flexible server allows for up to seven days at a time.

Additional features

The trek listing is cached for 60 seconds and cleared whenever a trek changes. Caching is skipped for admin and staff, because the cache key does not include the role and an admin request would otherwise put unapproved treks into the listing a trekker sees. The admin dashboard draws a Chart.js bar chart of the most popular treks. The reminder job can post to Google Chat. An admin can record payment on a booking. Forms use HTML5 validation with matching checks in the API, and the Bootstrap layout works at phone width. A `flask seed-demo` command loads staff, treks, trekkers and bookings so the application can be shown straight away.